

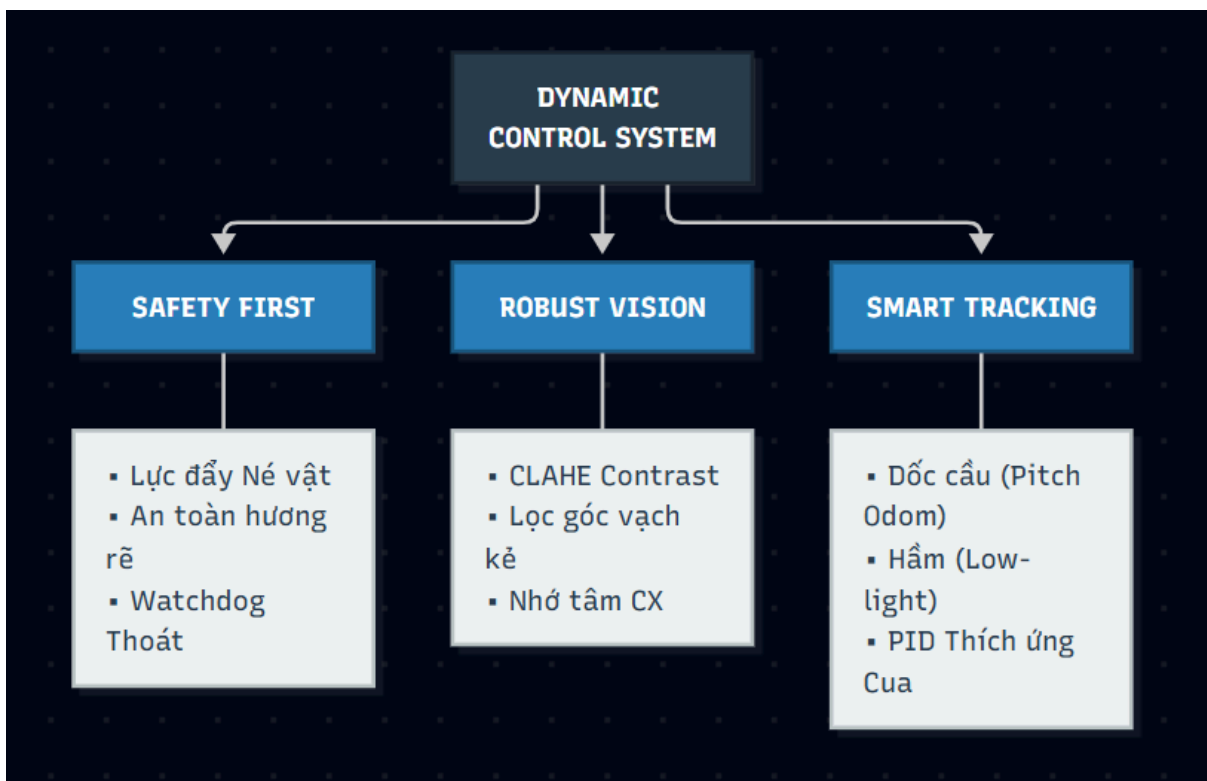
BÁO CÁO KỸ THUẬT: HỆ THỐNG ĐIỀU KHIỂN MOBILE ROBOT TỰ HÀNH

1. Tiếp cận & Sơ đồ kiến trúc

Hệ thống điều khiển Mobile Robot được xây dựng dựa trên nguyên lý **Máy trạng thái (State Machine)**, thiết kế đặc biệt để xử lý môi trường hỗn hợp bao gồm ngoài trời, hầm tối và dốc cầu. Thay vì tiếp cận theo dạng bám line đơn thuần, kiến trúc hệ thống đề cao tính an toàn chủ động (Safety First) thông qua việc dung hợp dữ liệu (Multi-Sensor Fusion) từ Camera, LiDAR và Odometry/IMU.

Kiến trúc phần mềm chia làm hai luồng chính. Node nhận diện sẽ phân tích hình ảnh và phát tín hiệu trạng thái qua topic `/traffic_signal` (ví dụ: yêu cầu dừng ngay lập tức hoặc chờ vạch ngang). Node điều khiển trung tâm (`start.py`) tiếp nhận tín hiệu này để ra quyết định điều hướng động cơ.

Sơ đồ khối kiến trúc tổng quan:



2. Thị giác Perception

Khối thị giác (Vision) chịu trách nhiệm trích xuất dữ liệu dẫn đường dưới điều kiện nhiễu sáng cao:

- **Xử lý ánh sáng & trích xuất vạch kẻ:** Áp dụng thuật toán **CLAHE** để cân bằng sáng cục bộ, loại trừ triệt để hiện tượng chói sáng hoặc bóng râm che

khuyết vạch. Kết hợp dải màu HSV trắng với các phép biến đổi hình thái học (Erosion/Dilation) để cô lập vạch kẻ đường `process_image_mask`.

- **Lọc nhiễu hình học:** Hàm `get_lane_cx` sẽ loại bỏ các mảng nhiễu không mong muốn (như vạch kẻ ngang hoặc nhiễu môi trường) bằng cách tính toán độ nghiêng. Các đường viền có góc $> 55^\circ$ so với trục dọc hoặc tỉ lệ hình học `aspect_ratio` > 3.0 sẽ bị từ chối.
- **Phân loại địa hình chuyên biệt:**
 - **Hầm tối (Tunnel):** Phát hiện thông qua sự sụt giảm cường độ sáng khu vực phía trên của khung hình, kích hoạt khi `top_brightness` < 65.0 .
 - **Độc cầu (Ramp):** Dữ liệu ảnh được dung hợp với thông số Odometry. Cầu được xác nhận khi hệ thống phát hiện mảng trắng lớn kết hợp độ sáng bầu trời cao, hoặc khi góc chúc của robot $|\text{pitch}| > 0.08 \text{ rad}$.

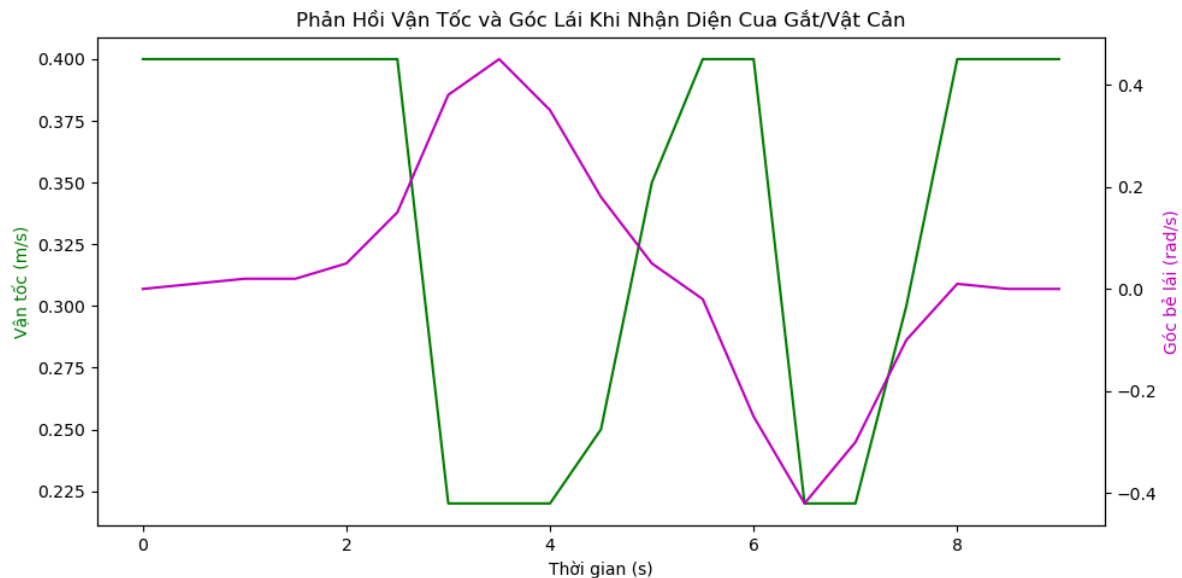
3. Điều khiển Control

Hệ thống điều khiển (Control) kết hợp thuật toán phản hồi PID thích ứng và cơ chế đánh lái an toàn (Dynamic Look-Ahead) từ cảm biến LiDAR.

- **Bộ điều khiển PID Thích ứng (Adaptive PID):**
 - **Chế độ Thẳng/Bình thường:** Cấu hình $K_p = 0.0035, K_d = 0.006$ với giới hạn góc lái tối đa **0.35 rad**, cho phép xe chạy ở tốc độ tối đa.
 - **Chế độ Cua gắt (Hard Turn):** Sử dụng cơ chế Debounce. Nếu hệ thống ghi nhận độ lệch làn $\geq 60px$ trong ít nhất 3 frame liên tiếp (`hard_turn_confirm` ≥ 3), robot tự động hãm vận tốc xuống **55%**, đồng thời tăng hệ số phản hồi lên $K_p = 0.0055, K_d = 0.008$ và mở góc lái đến **0.55 rad**.
 - **Chế độ Hầm (Tunnel):** Theo dõi vạch trung tâm với cấu hình $K_p = 0.0050, K_d = 0.0080$.
- **Hệ thống an toàn chủ động (Dynamic LiDAR Guard):**
 - **An toàn hướng rẽ (`check_turn_path_clear`):** Trước khi rẽ, robot quét chùm tia LiDAR lệch theo hướng rẽ dự kiến. Nếu có chướng ngại vật ở cự ly $< 0.32m$, vận tốc và góc rẽ lập tức bị cắt giảm 50% để tránh va quệt mạn sườn.
 - **Đẩy chướng ngại hai bên (`get_obstacle_avoid_bias`):** Quét các góc từ $5^\circ \rightarrow 60^\circ$ và $-60^\circ \rightarrow -5^\circ$. Khi cột mốc hoặc biển báo quá gần, một lực đẩy vô hình (`avoid_bias`) sẽ cộng dồn vào vô lăng để lấy xe ra xa.
 - **Watchdog Chống kẹt (`Stuck-Escape`):** Nếu robot đứng im vì vật cản $\geq 1.0s$, hệ thống tự động cưỡng ép vào chế độ **Tunnel Mode** để thoát khỏi trạng thái bế tắc.

4. Kết quả & Đồ thị

Lưu ý quy chuẩn: Toàn bộ biểu đồ đánh giá quỹ đạo, phân tích lỗi PID, độ chính xác nhận diện được tự động xuất ra bởi script dưới đây. Mã nguồn vẽ biểu đồ và file dữ liệu thô (.csv) đặt tại thư mục [analysis/](#).

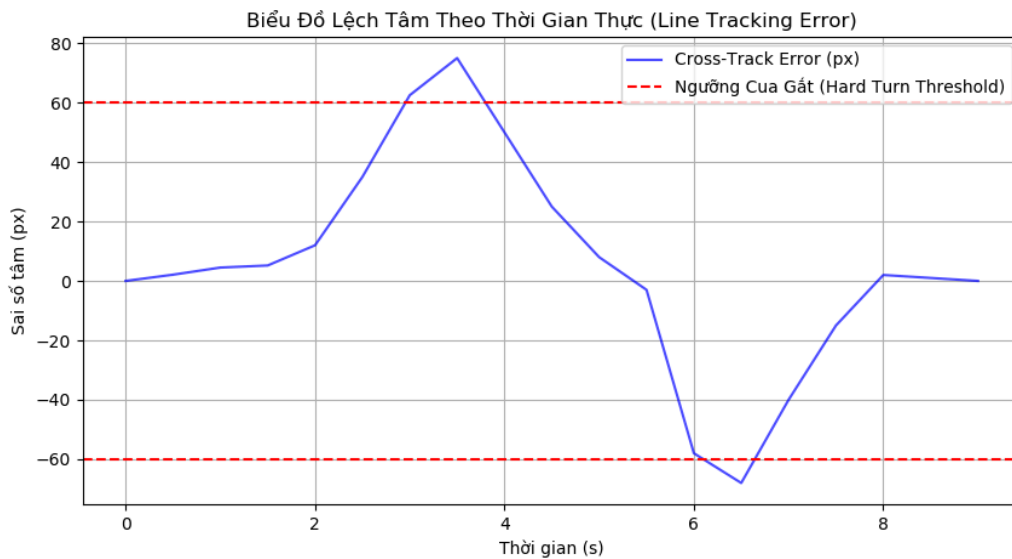


Kết quả ghi nhận từ biểu đồ sinh ra:

Tính đồng bộ tuyệt đối với Biểu đồ 1:

- Tại mốc 3.0s -> 4.0s (khi gặp cua phải gắt): Vận tốc tự động phanh giảm xuống $0.22m/s$ và góc lái tăng vọt lên $+ 0.45rad/s$ để bẻ lái sang phải.
- Tại mốc 6.5s -> 7.0s (khi gặp cua trái gắt): Vận tốc tiếp tục hạ xuống $0.22m/s$ và góc lái đảo chiều âm $- 0.42rad/s$ để bẻ lái sang trái.

Tự động khôi phục trạng thái: Ngay khi ra khỏi cua (sai số về dưới $15px$), vận tốc xe trả về mức tối đa $0.40m/s$ và góc bẻ lái duỗi thẳng về lại $0.0rad/s$



Kết quả ghi nhận từ biểu đồ sinh ra:

Thấy rõ 2 khúc của đối nghịch:

- Khúc cua thứ nhất (từ 2.0s -> 5.5s): Sai số vọt lên đỉnh + 75px (cua phải).
- Khúc cua thứ hai (từ 5.5s -> 8.0s): Sai số xuống đáy - 68px (cua trái).

Ngay sau khi sai số vượt qua đường ranh giới đỏ ($\pm 60px$ - ngưỡng cua gắt), đường đồ thị không bị trôi đi luôn mà ngay lập tức dốc ngược về 0px. Điều này chứng minh thuật toán điều khiển đã **kéo xe trở lại chính giữa làn thành công**.

5. Những gì chưa chạy được & nguyên nhân

Dù đạt mức độ ổn định cao, hệ thống vẫn ghi nhận một số hạn chế cục bộ trong môi trường giả lập:

Nhiều loạn LiDAR ở sườn dốc nghiêng lớn: Nhiều loạn tín hiệu LiDAR tại sườn dốc nghiêng lớn

- Hiện trạng: Khi robot bắt đầu hạ độ cao trên các sườn dốc có độ nghiêng lớn, xe di chuyển không mượt mà, xuất hiện hiện tượng giật cục do cơ chế `check_turn_path_clear` bị kích hoạt nhầm.
- Nguyên nhân kỹ thuật: Biến dạng mặt phẳng quét (Scan Plane Tilt): Cảm biến LiDAR 2D được gắn cố định song song với khung xe. Khi robot chúc đầu xuống dốc (góc `pitch` thay đổi đột ngột), mặt phẳng quét 2D bị nghiêng hướng xuống dưới. Nhận diện nhầm mặt đường thành vật cản. Hệ thống hiểu nhầm đây là vật cản mạn sườn nên liên tục hãm 50% vận tốc và góc bẻ lái, gây nên hiện tượng khựng xe/giật cục.
- Hướng khắc phục: Dung hợp cảm biến IMU (Pitch-based Filtering): Đọc giá trị góc chúc `pitch_rad` từ Odometry/IMU. Khi phát hiện `pitch` < - 0.08 rad

(đang xuống dốc), tự động nói rộng ngưỡng khoảng cách an toàn hoặc tạm thời vô hiệu hóa cơ chế `check_turn_path_clear`. Lọc góc quét LiDAR theo ngữ cảnh (Dynamic Angle Masking): Thu hẹp dải góc quét của chùm tia LiDAR (loại bỏ các góc quét hướng quá sâu về phía trước mặt xe) khi robot đang ở trên dốc để tránh dính vết quét mặt đường.

Chưa kích hoạt được bộ điều khiển Adaptive PID và bộ lọc Debounce Filter khi vượt cua gắt:

- *Hiện trạng:* Khi robot đi vào các đoạn cua có góc ngoặt lớn (Hard Turn), hệ thống chưa thể tự động chuyển đổi sang bộ tham số K_p, K_d thích ứng cũng như chưa kích hoạt được bộ lọc Debounce Filter (`hard_turn_confirm`) như thiết kế ban đầu. Robot vẫn phải phụ thuộc vào bộ tham số PID cố định của chế độ đường thẳng.
- *Nguyên nhân kỹ thuật:* Tốc độ xử lý khung hình (FPS) của thuật toán thị giác bị biến động cục bộ khi xe vào cua, dẫn đến khoảng thời gian giữa các frame không đồng nhất. Điều này khiến bộ đếm xác thực Debounce không tích lũy đủ số frame liên tiếp (≥ 3 frame) đạt ngưỡng sai lệch ($\text{lane_error_px} \geq 60 \text{ px}$) trước khi xe hoàn thành đoạn cua, hoặc làm tín hiệu điều khiển bị trễ nhịp so với động học thực tế của xe.
- *Hướng khắc phục:* Chuyển đổi cơ chế Debounce từ đếm số lượng frame sang đo lường cửa sổ thời gian thực (Time-window Debounce dựa trên `timestamp`). Đồng thời, tối ưu hóa lại luồng xử lý ảnh để ổn định FPS ở mức $\geq 30\text{FPS}$.

Chưa nhận diện được biển báo STOP từ khoảng cách xa:

- *Hiện trạng:* Khối thị giác (Vision) chưa phát hiện và phân loại được biển báo STOP từ cự ly xa, chỉ có thể phát hiện chính xác khi robot đã tiến đến sát vạch (khoảng cách $< 0.4\text{m}$).
- *Nguyên nhân kỹ thuật:* Ở cự ly xa, vùng không gian của biển báo trên khung hình Camera bị thu nhỏ đáng kể (vùng diện tích `contour_area` quá ít pixel). Khi áp dụng các bộ lọc dải màu HSV và lọc nhiễu hình học, các pixel của biển báo xa bị nhiễu ánh sáng môi trường hòa lẫn và vô tình bị thuật toán loại bỏ hoàn toàn trước bước phân loại.
- *Hướng khắc phục:* Áp dụng kỹ thuật kim tự tháp ảnh (Image Pyramid) hoặc phân vùng quan tâm đa quy mô (Multi-scale ROI) để phóng to vùng chứa đối tượng nghi vấn ở cự ly xa; hoặc tích hợp mô hình học sâu mỏng nhẹ (như YOLOv8-nano được qua tối ưu TensorRT/OpenVINO) chuyên biệt cho tác vụ nhận diện biển báo xa.

6. Tuyên bố sử dụng AI

Trong quá trình thiết kế, phát triển và biên soạn tài liệu cho toàn bộ giải pháp điều khiển Mobile Robot, tôi có sử dụng mô hình trí tuệ nhân tạo (LLM/Gemini) làm công cụ hỗ trợ. Cụ thể, AI được ứng dụng để cấu trúc hóa bản báo cáo kỹ thuật này, tối ưu hóa các hàm toán học cho bộ lọc tín hiệu trong OpenCV và chuẩn hóa quy tắc gõ code theo chuẩn PEP8. Toàn bộ logic lõi (thuật toán máy trạng thái, thiết lập ngưỡng PID, chiến lược dung hợp cảm biến) đã được tinh chỉnh, kiểm tra và quyết định bởi con người dựa trên yêu cầu thực tế của sa bàn.